```
x = %w[these are all words]
x.sum { |word| word.length }      # => 16
x.map { |word| word.length }.sum  # => 16
```

Another common pattern for reduce behavior is to build up a new container from the data in an existing one, for example, a hash based on the data in an array. Ruby provides each_with_object as a shortcut for this pattern:

```
x = %w[bananas are the funniest fruit]
result = x.each_with_object({}) do |word, result|
  initial = word[0]
  result[initial] ||= []
  result[initial] << word
end
puts result
```

produces:

```
{"b"=>["bananas"], "a"=>["are"], "t"=>["the"], "f"=>["funniest", "fruit"]}
```

The basic idea of each_with_object is the same as reduce but there are two differences: the order of the block arguments is [element, accumulator] rather than [accumulator, element], and what is passed from iteration to iteration is the accumulator itself, rather than the return value of the block. In this case, the two are different, and the return value of the block is word.

A few kinds of object creation techniques are are common enough to get their own methods. The group_by method is a generalization of what we just did: converting a list to a hash based on the value of some block. So we could write our initializer like so:

```
x = %w[bananas are the funniest fruit]
result = x.group_by { |word| word[0] }
puts result
```

produces:

```
{"b"=>["bananas"], "a"=>["are"], "t"=>["the"], "f"=>["funniest", "fruit"]}
```

In this case, the existence of the resulting hash and the accumulation of the values is handled by group_by.

The partition method takes a block and returns a two-element array of arrays. The first element of the result is an array of all entries in the initial list for which the block returns a truthy value. The second is the elements for which this is not so:

```
x = [1, 2, 3, 4, 5]
x.partition { |z| z.even? } # => [[2, 4], [1, 3, 5]]
```

The zip method combines a list with one or more other lists to create a new array of arrays, where each element it the result is made up of the corresponding element of the original list and all the arguments, like this:

```
foo = [1, 2, 3, 4, 5]
bar = ["a", "b", "c", "d", "e"]
foo.zip(bar)  # => [[1, "a"], [2, "b"], [3, "c"], [4, "d"], [5, "e"]]
```

Filtering

Ruby provides a wide variety of ways to find a particular object or objects in a collection along with the related task of filtering a collection based on some arbitrary criteria.